

答疑课堂（二） | 第二章Rust进阶篇思考题答案

唐刚 · Rust 语言从入门到实战



你好，我是 Mike。

这节课我们继续来看第二章的课后思考题答案。还是和之前一样，最好是自己学完做完思考题之后再来看答案，效果会更好。话不多说，我们直接开始吧！

进阶篇

🔗 12 | 智能指针：从所有权和引用看智能指针的用法

思考题

你试着打开示例中的这两句，看看报错信息，然后分析一下是为什么？

```
1 // arced.play_mutref(); // 不能用
2 // arced.play_own();    // 不能用
```

 复制代码

答案

Arc 本质上是个引用，所以不允许同时存在可变引用或者移动。play_boxown() 和 play_own() 只能同时打开一个，这两个方法调用都会消耗所有权，导致没法调用另外一个。

答案来自 Taozi 和 Michael

🔗 13 | 异步并发编程：为什么说异步并发编程是 Rust 的独立王国？

思考题

为什么我们要把 async Rust 叫做“独立王国”呢？

答案

因此 async Rust 代码是在一个 Runtime 里面执行的，而 std Rust 的代码不需要这个额外的 Runtime，因此说它是独立王国。

另一方面，在 Rust 中，异步编程是使用 async/await 语法，这种语法具有可传染性，与 std Rust 代码也可以明显区分开，因此它像一个独立王国。

🔗 14 | Tokio 编程（一）：如何编写一个网络命令行程序？

思考题

1. EOF 是什么，什么时候会碰到 EOF？
2. `stream.read_to_end()` 接口能读完网络连接中的数据吗？

答案

EOF 是 End of file。在 Linux 万物皆 file 的情况下，connection 也可以是一个 file。所以，当 Connection 关闭的时候，就会产生 EOF。

`stream.read_to_end()` 是持续 `read()` 直到 EOF，因此能够读完网络里的数据，如果使用 `stream.read_to_end(&mut buf).await?` 读取的话，会持续 wait，直到连接关闭

才能进行后续的操作。

答案来自 PEtFiSh

🔗 15 | Tokio 编程（二）：如何在 Tokio 多任务间操作同一片数据？

思考题

下面这两句的意义是什么，第一行会阻塞第二句吗？

```
1 _ = task_a.await.unwrap();  
2 _ = task_b.await.unwrap();
```

 复制代码

答案

await 代码会持续等待直到任务结束，因此在 main thread 里第一行会阻塞第二行。但这不会让 task_a 阻塞 task_b。加入 await 可以使最后的 println! 打印两个任务执行完以后被修改的 db 值，如果不加入 await。有一定几率最后 println! 打印的还是原始的 db。

答案来自 PEtFiSh

🔗 16 | Tokio 编程（三）：如何用 channel 在不同任务间进行通信？

思考题

从任务中搜集返回结果有几种方式？

答案

从任务收集返回结果的方式有：

1. 任务直接返回值，然后通过 handler 取回，比如 `a = task_a.await.unwrap();`。
2. 通过锁的方式直接写在目标位置

3. 通过 channel 的形式传递结果
4. 似乎也可以 unsafe 来写全局变量。

答案来自 PEtFiSh

🔗 17 | Tokio 编程（四）：Rust 异步并发还有哪些需要关注的点？

思考题

你是如何理解 “async Rust 是一个独立王国” 这种说法的？

答案

略

🔗 18 | 错误处理系统：Rust 中错误是如何被传递并处理的？

思考题

1. 请你查阅 Rust std 资料，并说说对 `std::error::Error` 的理解。
2. 请说明 `anyhow::Error` 与自定义枚举类型用作错误的接收时的区别。

答案

`std::error::Error` 是 Rust 标准库中的 trait，该 trait 表示可能的错误类型的共同行为，这些错误类型通常都表示了一个失败的操作或无效的输入。实现 Error trait 的类型必须提供一个错误描述和一个可操作性的字符串，通常用于打印错误信息。

`anyhow::Error` 是一个第三方 Rust 库，它提供了一种用于简化错误处理的方式，该库提供了一个 Error 类型，能够自动收集多个错误信息，并返回一个更易于调试和处理的错误类型。

与自定义枚举类型相比，使用 `anyhow::Error` 具有以下优点：

1. 多态性：`anyhow::Error` 是一个特殊的错误类型，可以表示任何可能失败的操作，并且可以方便地传递错误信息和上下文，从而使错误处理更加灵活和方便。

2. 方便处理：anyhow::Error 提供了一组常用的操作，如错误信息的格式化和日志记录等，这些操作相对繁琐，如果使用自定义枚举类型实现会相对麻烦和复杂。
3. 减少代码重复：使用 anyhow::Error 可以排除许多写错误处理代码的繁琐重复的代码，提高代码可读性和可维护性。

而在使用自定义枚举类型来接收错误时，你需要眼睛注意只能返回对应的错误类型，因此可能需要对函数中的一些操作手动类型转换（map_error）。不像 anyhow::Error 这样可以一股脑直接扔回去。也就是说使用自定义枚举类型来接收错误的心智负担会大一点。

🔗 19 | Rust 的宏体系：为自己的项目写一个简单的声明宏

思考题

说一说 allow, warn, deny, forbid 几个属性的意义和用法。

答案

在 Rust 中，allow、warn、deny 和 forbid 是四个控制编译器警告和错误输出的属性。

1. allow：允许代码中出现这些警告，编译器不会输出警告信息。
2. warn：将警告转换为编译器的错误级别，编译器会在警告信息后面输出错误信息，但不会阻止代码的编译。
3. deny：将警告变为编译错误，编译器会在错误信息后面输出警告信息，并且由于错误级别的提高，代码无法编译通过。
4. forbid：严格禁止代码中出现这些警告，如果代码中出现了这些警告，就会直接报错并停止编译。

这些属性通常用于优化编译器输出，使输出更加简洁和准确，并降低代码中不良代码的风险。一般情况下，allow 属性用于代码库中的不重要代码部分，warn 用于需要注意但不会造成严重后果的代码部分，deny 用于那些可能会导致问题的代码部分，而 forbid 则用于绝对不能出现问题的代码部分。

例如，在 Rust 中可以这样使用这些属性：

[复制代码](#)

```
1  #[allow(dead_code)]
2  fn unused_function() {
3      // some unused code
4  }
5  #[warn(unused_variables)]
6  fn unused_variable() {
7      let _unused_var = "unused variable";
8  }
9  #[deny(unused_imports)]
10 use std::collections::HashMap;
11 #[forbid(unsafe_code)]
12 fn safe_function() {
13     // some safe code that is not allowed to be unsafe
14 }
```

在代码中使用 allow、warn、deny 和 forbid 属性可以帮助开发者更加简单高效地控制代码的输出和行为，并帮助代码更好地遵循 Rust 语言特性和最佳实践。

🔗 20 | 生命周期：Rust 如何做基本的生命周期符号标注？

思考题

你能说一说生命周期符号 'a 放在 `<>` 中定义的原因和意义吗？

答案

因为 'a 符号本来就是一种 generic 符号。<T, 'a> 中的类型参数 T 用来代表编译期空间上的分析展开，'a 用来代表编译期时间上的分析展开。所以，'a 放在 <> 号中是理所当然的。它们的存在都是为了给编译器提供多一些信息。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

精选留言

由作者筛选后的优质留言将会公开显示，欢迎踊跃留言。